biohub-cell-tracking-during-development

14th place solution from Vibes&Edges Trade-Off

Rank	14
Team	Vibes & Edges Trade-Off
Author	Tom
Topic ID	744486
Version	Original

Source: https://www.kaggle.com/competitions/biohub-cell-tracking-during-development/discussion/744486
Retrieved with Kaggle CLI on 2026-10-04.

Thank host announce such amazing competition. Here is the solution writeup from Vibes&Edge Trade-Off.

TL;DR

We run **two independent tracking pipelines over the same volumes and reconcile them per video**, rather than tuning one.

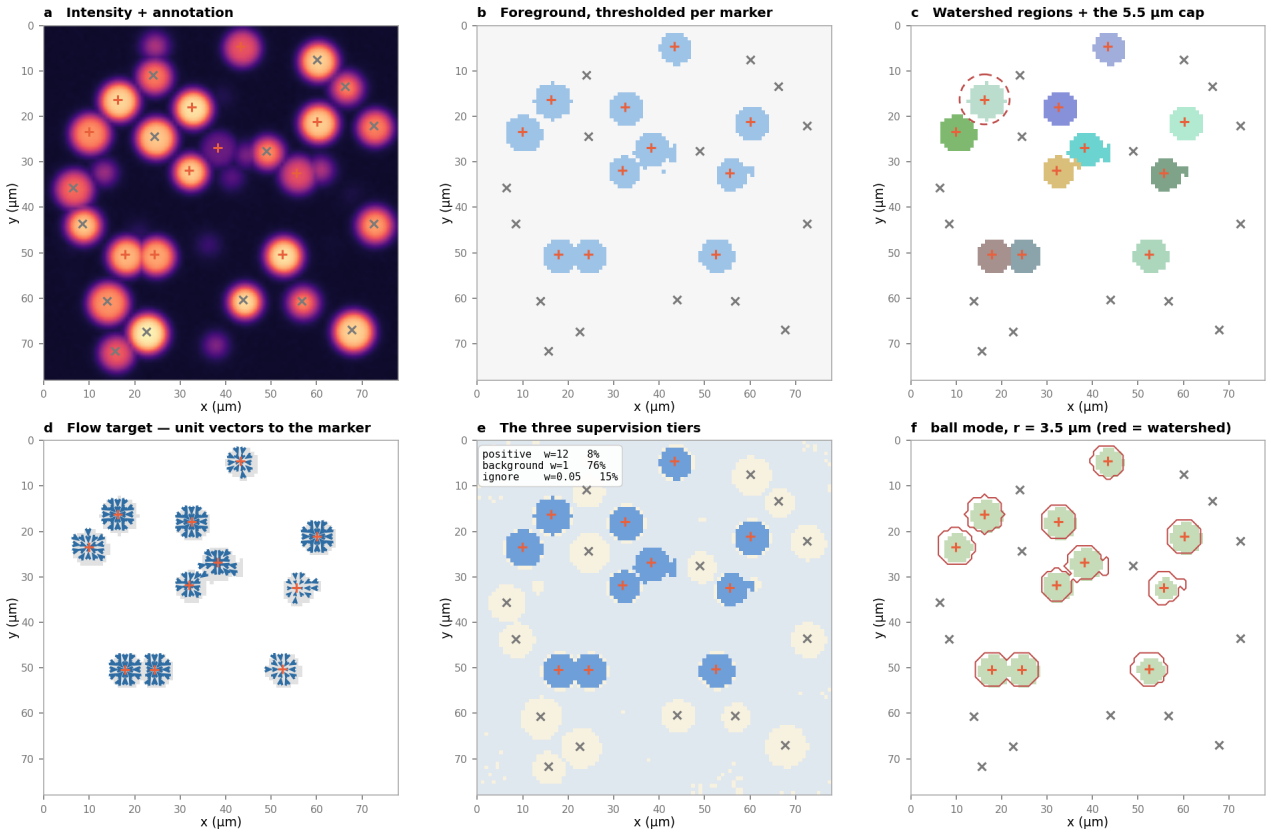
- **Detection.** A flow-field segmenter (FlowSeg) predicts, for every voxel, a vector pointing at its own cell centre; integrating that field separates touching nuclei that no threshold can split. A second, differently-seeded TemporalUNet3D detector runs alongside it so the two fail in different places.
- **Edge.** One pipeline links with a global ILP, the other with a 5-fold GNN over shape features. Gates are applied to the residual displacement after the whole frame's rigid motion is removed, and three gap closers recover cells the detector saw but did not believe.
- **Division.** Instead of classifying "is this a division?" from 121 labelled events, divflow **regresses correspondence**: every voxel of a later frame predicts where its parent was. A division is then a geometric coincidence — two instances converging in parent space while staying apart in observed space. Everything after that is filtering.
- **Merge.** The two graphs are grafted, not averaged, and a per-video circuit breaker falls back to the stronger graph when our detector collapses on a video.

The two decisions that moved the score most were not models: **linefit smoothing** of node positions (−0.01914 when off) and a **zero-parameter constant coordinate shift** applied after every graph decision (+0.002 on the leaderboard).

Detection

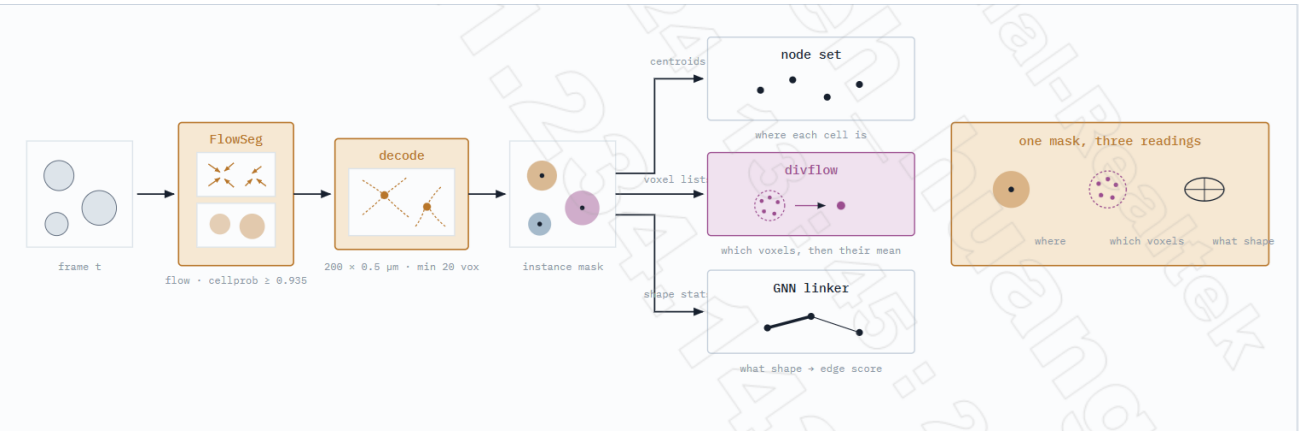
Flowseg

- The problem: These volumes are densely packed nuclei. Any method that segments by thresholding a probability map fails at exactly the place that matters: two touching cells have no intensity valley between them, so they merge into one blob. Predicting boundaries directly is worse in 3D: a boundary is a thin, low-mass target, and one gap in it merges two cells.
- The idea: predict a field, not an object. For every voxel, the network regresses a vector pointing toward the centre of the cell that voxel belongs to. Segmentation then stops being a labelling problem and becomes a dynamical one: release each foreground voxel and integrate along the field. Voxels of the same cell converge on the same attractor; voxels of different cells converge on different ones. The separation is carried by the direction of the field, not by contrast in the image, so two cells that physically touch are still separated cleanly: at their shared surface the flows point in opposite directions, a discontinuity a threshold cannot see but an integrator resolves immediately.



- How it is realised: cellprob ≥ 0.935 decides which voxels get released; the decoder takes 200 steps of 0.5 µm; voxels that land on one sink form one instance; instances under 20 voxels are discarded. Four flip views are decoded independently and their sinks merged by consensus, so a sink that only one view produces does not survive.

The part that is easy to miss. The output is not "a segmentation" that gets consumed once. The same mask is read three different ways by three different modules: its centroids become the node set for tracking, its per-instance voxel membership is the support that divflow averages over, and its shape statistics (volume, equivalent radius, the three eigenvalues) become node features for the GNN linker. One forward pass, three distinct downstream meanings.



Hungarian Sinking and Intersection with UNet

The detector is run twice with two FlowSeg checkpoints that share an architecture but are trained on different temporal evidence. sym (diff_mode="symmetric") sees a 3-frame centred window [t−k, t, t+k] and feeds its diff

encoder a single central difference. stack4 (diff_mode="stack4_mixed") sees a 5-frame window [t−2k, t−k, t, t+k, t+2k] and stacks four differences on the channel axis: two symmetric spans (t+k − t−k, t+2k − t−2k) and two asymmetric cross-spans (t+k − t−2k, t+2k − t−k). A short symmetric window is sharp on cells that move little between frames; the mixed 5-frame stack carries motion over a longer baseline and survives frames where the short difference is dominated by noise. They fail on different cells, which is the whole reason for running both.

Each model decodes its own flow field to sinks independently, per frame. The two sink sets are then merged by a Hungarian consensus rather than a proximity vote. For each frame we build the full pairwise distance matrix in micrometres (voxel scale (1.625, 0.40625, 0.40625)), price every pair beyond r = 6.0 µm out of contention, solve a one-to-one assignment, and discard whatever the solver was still forced to take above the radius. A sink only accumulates a vote if it is the other model's globally optimal exclusive partner — not merely its nearest neighbour.

That exclusivity is the point. The earlier merge was a soft inverse-distance vote, which is many-to-one: when one model splits a single cell into two fragments, both fragments can collect a vote from the same sink in the other model, and both survive. This pipeline's dominant failure mode is over-detection, so a rule that lets one piece of evidence back an unbounded number of detections is the wrong rule. Under the Hungarian, one sink backs at most one sink.

The merge is run twice with each model as the anchor, because the assignment always accumulates onto the first list's sinks and therefore cannot create a cluster the anchor never proposed. The two results are unioned and deduplicated at 4.0 µm. The quorum is $0.75 \times$ n_entries; with two entries that rounds up to unanimity, so a detection must be seen by both models to survive. The surviving position is the mean of the matched pair, which is where the fusion also buys a little localisation.

unettf as a veto, not a vote

The fused FlowSeg node set then passes through arm-D intersection with a completely separate detector, unettf — a different architecture, different normalisation, different training script. Per frame we query a KD-tree of unettf detections and keep a FlowSeg node if any unettf detection lies within INTERSECT_UM = 6.0 µm.

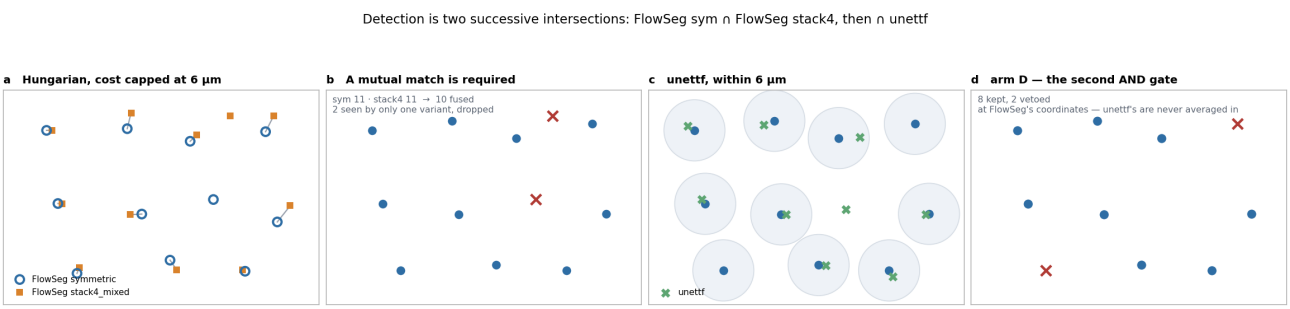
Two properties of this stage matter more than they look:

The kept node keeps FlowSeg's coordinates. unettf's positions are never averaged in. FlowSeg localises better, and every attempt at mixing the two coordinate sources measured negative.

unettf can only veto, never vote, and never seed. A cell that only unettf sees does not enter the graph. This is deliberately asymmetric — unettf's job is precision, not recall, and admitting its unique detections was measured as a loss. For the same reason unettf's folds are fused by an elementwise logit mean before peak-finding, not by concatenating each fold's peaks: a union of peaks would make the arm-D gate strictly easier to pass with every fold added, which is the opposite of what a corroboration gate should do.

We kept the many-to-one form of arm-D here even though the same exclusivity argument applies. The Hungarian and mutual-nearest-neighbour variants were both built and measured; mutual scored 0.89730 against arm-D's 0.89980 — it did cut division FP from 12 to 10, but cost a division TP (15→14) and dropped node recall from 0.9652 to 0.9579, because reciprocal matching throws away a perfectly good pair whenever the unettf node's own nearest happens to be someone else. That is a local test on a global problem, and at this stage the recall loss was not worth the precision.

The net shape is a two-stage corroboration: two FlowSeg views must agree with each other exclusively, and the survivors must then be independently witnessed by a different architecture. Everything downstream — the GNN linker, divflow, the repair layers, filter_output_graph — runs unchanged on the resulting node array.



Dual-seed TemporalUNet3D

- The problem: FlowSeg is one opinion about where the cells are. A single detector's misses are systematic rather than random — it drops the same dim, crowded or edge-of-volume cells in every frame of a video, and a tracker cannot recover a cell that was never detected. What is wanted is a second detector whose failures are differently shaped.
- The idea: a temporal U-Net that sees more than one frame at a time and emits a per-voxel detection heatmap, followed by a transformer that scores every (t, t+1) node pair by cross-attention. Cell centres are the local maxima of the heatmap, so the threshold and the non-maximum suppression a hand-tuned detector would apply are instead absorbed into the shape of the heatmap itself.
- Dual seed: the same architecture is trained twice under different seeds and both run at inference. Their candidate edges are fused by a harmonic probability rule (weight 0.15) that requires a pair to be supported in both the forward and the reverse pass, with a retention floor of 0.90 so the fusion can never quietly discard most of a frame's candidates.

Post-hoc coordinate correction

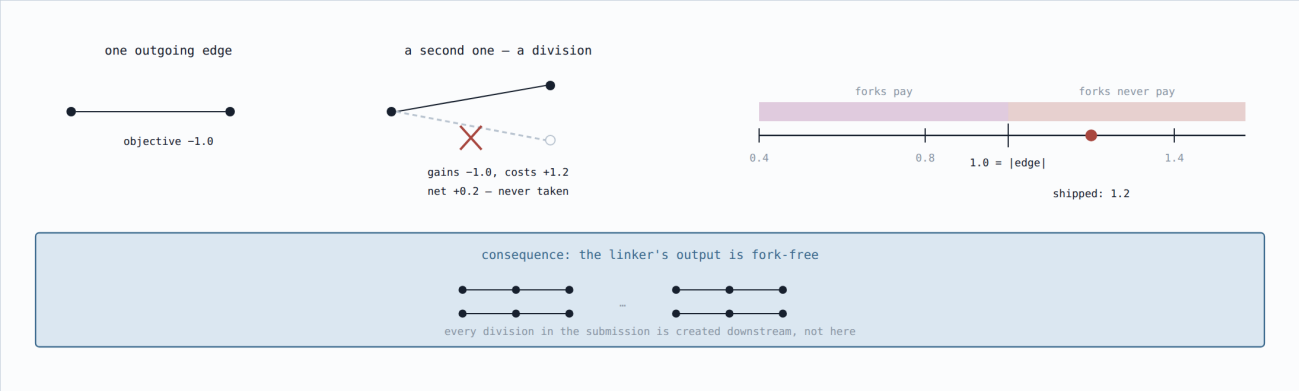
- The problem: our detections carry a small systematic offset from the annotation convention. Every learned correction head we tried bought its gain on the edge term by destroying a division.
- The idea: one constant vector, zero parameters. Estimated as the mean GT − prediction residual and fitted on folds 1–4 only, so the evaluation fold is never seen: `(+0.6807, +0.2067, +0.1725) µm`, which is `(+0.419, +0.509, +0.425)` voxels.
- Where it is applied, and why that is the whole point: **after every graph decision**. Measured both ways, the shift is worth **+0.01116 at the scorer** and **−0.00573 to the graph** — moving detections before linking makes the linker draw 73 fewer true edges. Applied last, only the good half is banked. On the leaderboard it was worth **+0.002**.

Edge

ILP linker

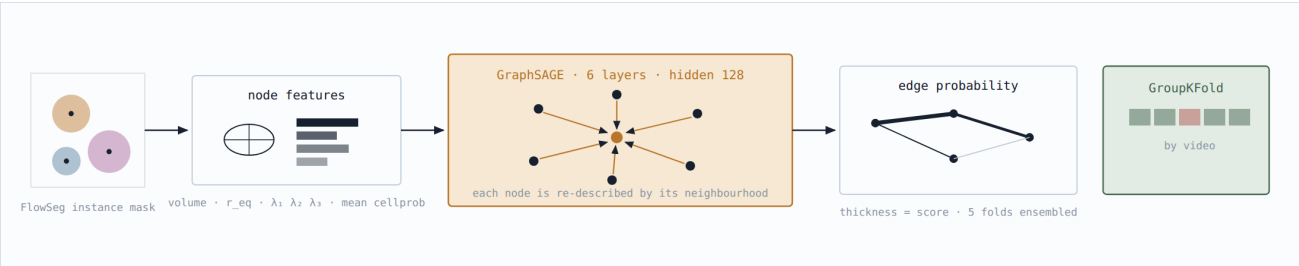
- The problem: given candidate edges with probabilities, choose the subset that forms a valid lineage forest. Greedy or Hungarian matching decides each frame pair in isolation, so a locally attractive wrong edge is never revisited.

- The idea: write the whole video as one integer program and let a solver take every decision at once. Each candidate edge carries a reward, appearing and disappearing carry costs, and a second outgoing edge from one node — a division — carries its own price.
- The part worth stating plainly: with `edge = −1.0` and `division = 1.2`, a second outgoing edge gains 1.0 and costs 1.2. The break-even sits exactly at `|edge| = 1.0`, so **at 1.2 a fork never pays and the ILP output contains zero divisions by construction**. That is why every division in our final submission is created by a later stage rather than by the linker. `appearance = 0.0` and `disappearance = 2` make ending a track expensive and starting one free, which biases the solver toward continuing a track through weak evidence rather than cutting it.



GNN linker

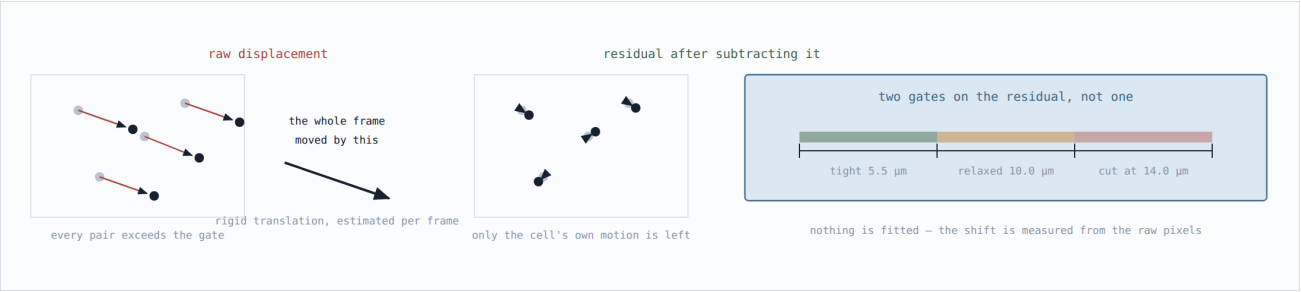
- The problem: the detector's edge head sees two nodes. It cannot see that a proposed edge would leave a neighbour stranded, or that the same target is being claimed by a better-matching source one cell over.
- The idea: build one graph per (video, frame) and classify with message passing, so each decision is made in the context of its neighbourhood instead of in isolation.
- How it is trained: GraphSAGE, hidden 128, 6 layers, `jk='cat'`, dropout 0.3, one graph per (video, frame), 5-fold GroupKFold **by video**. Node features are the FlowSeg mask's own shape statistics — volume, equivalent radius, the three covariance eigenvalues, mean cellprob — which is the third of the three readings of that mask.
- How it predicts: a probability per candidate edge, five folds ensembled.



Motion relink

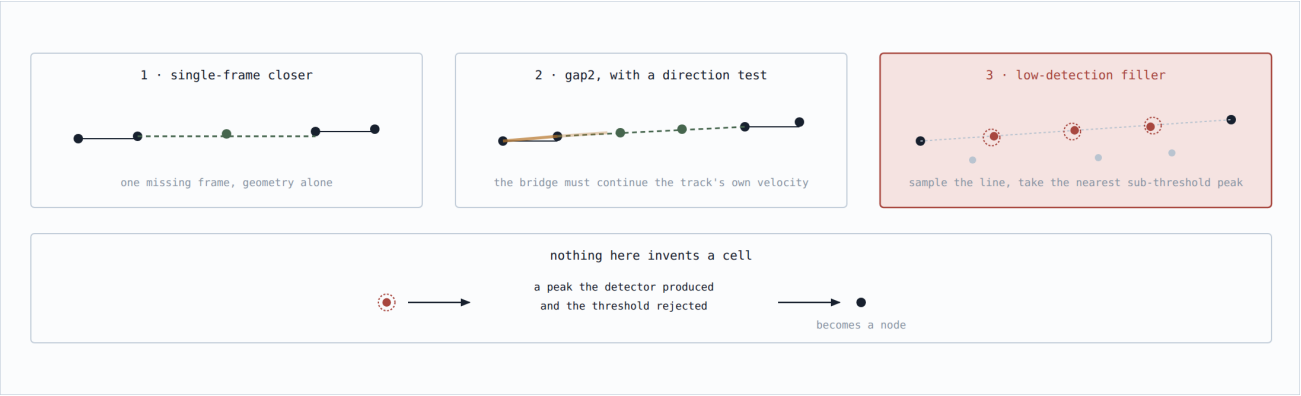
- The problem: the embryo moves. A gate expressed as "a cell cannot travel more than X µm between frames" is really measuring two things at once — the cell's own motion and the whole volume's drift — and the second term can dominate the first.

- The idea: estimate the rigid translation of the whole frame, subtract it, and gate on the residual. The gate then measures what it was meant to measure. Nothing is fitted; it is pure measurement from the raw pixels.
- How it predicts: two gates rather than one — a tight 5.5 µm for confident pairs and a relaxed 10.0 µm for the rest, with 14.0 µm as a hard ceiling. Edges spanning non-consecutive frames are removed outright.



Three gap closers, cheapest first

- The problem: a detector that misses a cell for one frame splits one track into two, and both halves then look like a birth and a death. The scorer charges for both.
- The idea: a single-frame closer joins an end at t to a start at t+2 when the geometry is unambiguous. A second pass handles two-frame gaps with a direction test against the track's own velocity. What is still open goes to the low-detection filler — the detector's sub-threshold peaks were dumped during prediction, and the straight line from the open end to the open start is sampled at each missing frame, taking the nearest free peak within the peak radius. A cell that was seen but not believed is much cheaper to recover than one that was never seen.
- Readmit: the same sub-threshold pool feeds a simpler rule. A discarded peak scoring at least 0.965 that sits within 4 µm of a node with no outgoing edge, or no incoming edge, is put back, and the graph is re-linked afterwards.
- Pruning: nodes left with no edges are deleted, and whole connected components shorter than 6 frames are deleted. A six-frame fragment is far more likely to be a detector artefact than a cell that entered and left the field of view.
- What it deliberately is not: none of these invent a cell. Every node added is a real peak the detector produced and the threshold rejected.

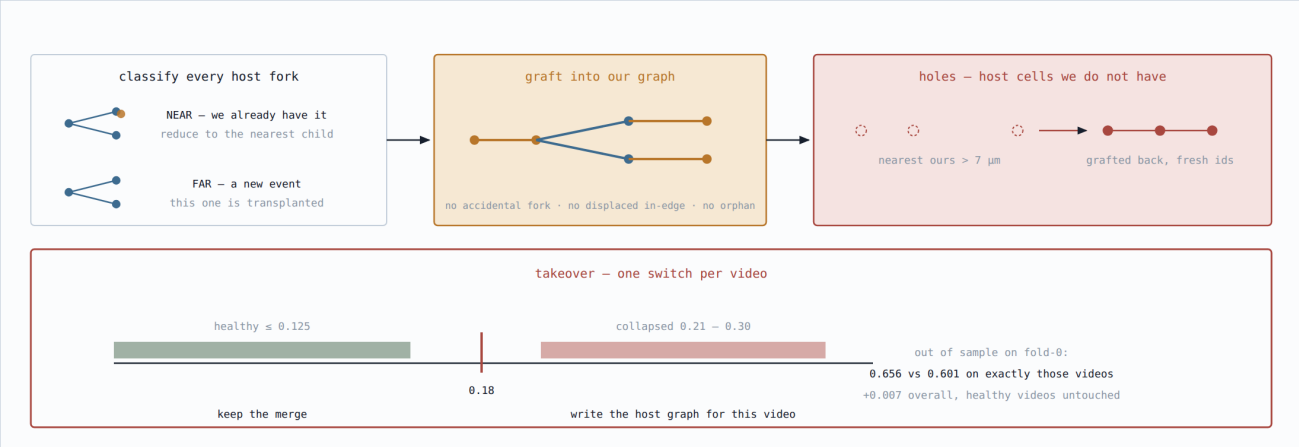


Linefit smoothing

- The idea: a node's position is measured independently in every frame, so a track inherits the detector's per-frame jitter. Refit each node to a straight line through its own neighbourhood and take the fitted point. Cells move smoothly at this frame rate, so the fit removes noise rather than signal.
- Why it gets its own heading: turning it off costs **−0.01914**, the single largest mechanism in our chain.

Merging two graphs

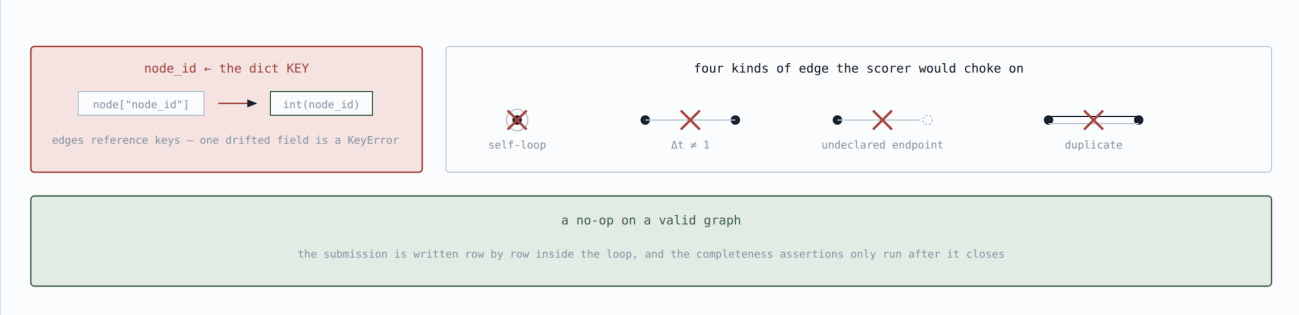
- The problem: two independent pipelines produce two complete track graphs of the same video. They disagree about which cells exist, which are linked, and where the divisions are. Neither is uniformly better.
- The idea — graft, do not average: averaging two graphs is not defined. One graph is the frame and the other contributes specific, named structures. Every fork the other graph holds is first classified as near — the same division event we already have — or far; near ones are reduced to their nearest child so the graft cannot duplicate an event; only far ones are transplanted. The transplant obeys three refusals: never create an accidental fork, never displace an in-edge of one of our original forks, never skip a segment in a way that orphans a node.
- Holes: a node of the other graph with nothing of ours within 7 µm in its own frame is a hole — a cell it found and we do not have. Those are grafted back with fresh ids. The same pass reports `hole_fraction = holes / its nodes`, which is our detector's deficit on that video.
- The takeover: on a few videos our detector collapses, and the hole fraction measures it directly — healthy fold-0 videos are all ≤ 0.125, collapsed ones 0.21–0.30. At ≥ 0.18 the merge is discarded entirely for that video and the other graph is written instead. Out of sample on fold-0 it wins exactly on those videos (0.656 vs 0.601, +0.007 overall) and never touches a healthy one. It is a fallback rather than a mechanism: the worst it can do is ship the baseline for a handful of videos.



Format guard

- The idea: the submission is written row by row inside the per-video loop, and the completeness assertions only run after that loop closes. So the last thing before the writer is a guard that removes exactly what the scorer would choke on — self-loops, endpoints that are not declared nodes, duplicate edges, and any edge whose frame delta is not 1. It also writes the dictionary key as `node_id` rather than the stored field, because edges reference keys and one drifted field is a `KeyError` inside the scorer's id map.

- Why a guard rather than a fix upstream: it is a no-op on a valid graph. Its cost is zero, and its value is that a rare, data-dependent inconsistency cannot reach the scorer.



Division

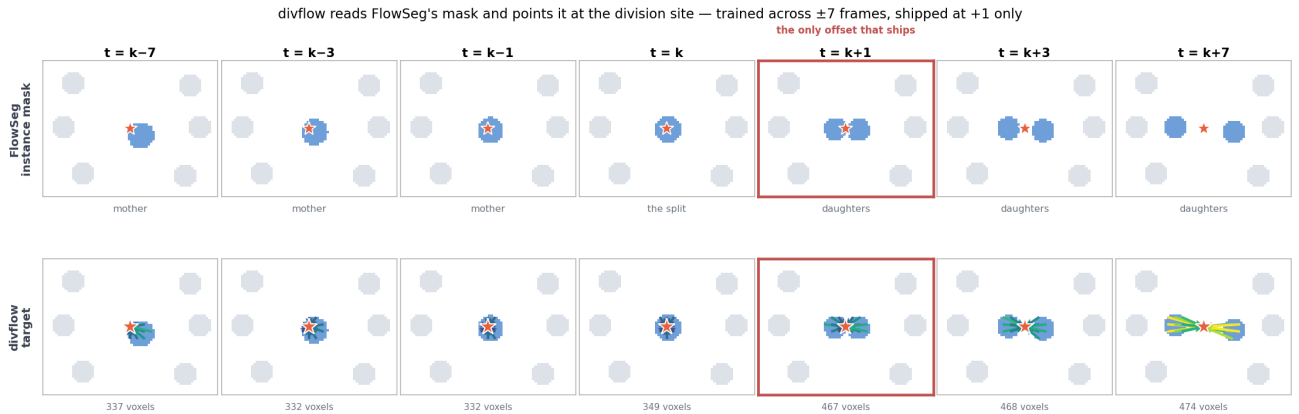
Divflow

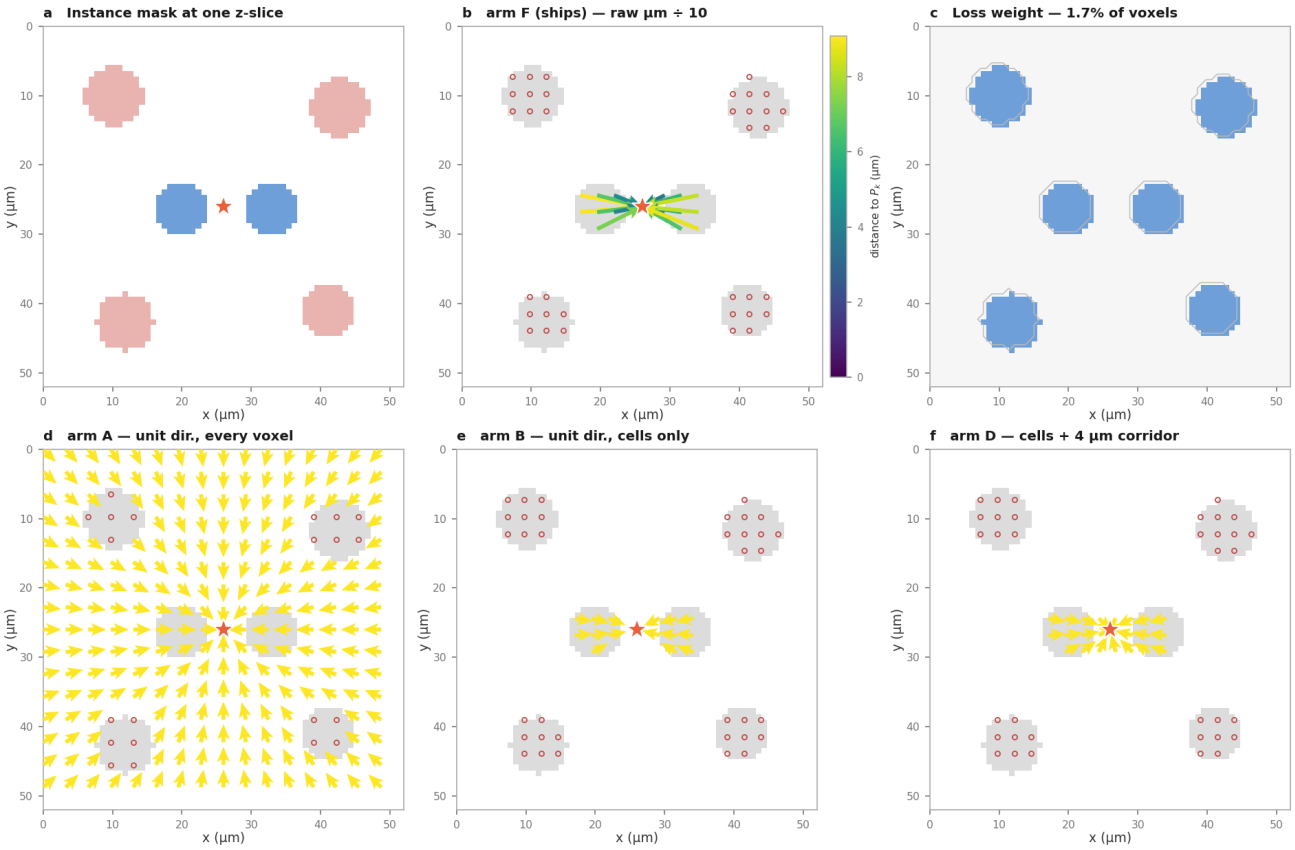
- The problem: Deciding whether a cell divided. Treated as classification: "is this a division?" it is a rare-event problem with very little supervision: the training set contains 121 real division events. A classifier trained on 121 positives against a large negative pool learns the negatives.
- The idea: regress correspondence, then let geometry decide. Instead of classifying divisions, the network predicts, for every voxel of a later frame, the displacement back to where its parent was. Division is then never classified at all. It is a coincidence that either holds or does not:

Convergence in parent space. Two instances in frame k+1 whose predicted parent positions fall within 4 µm of each other are claiming a common origin. Separation in observed space. Those same two instances must be 3–16 µm apart right now.

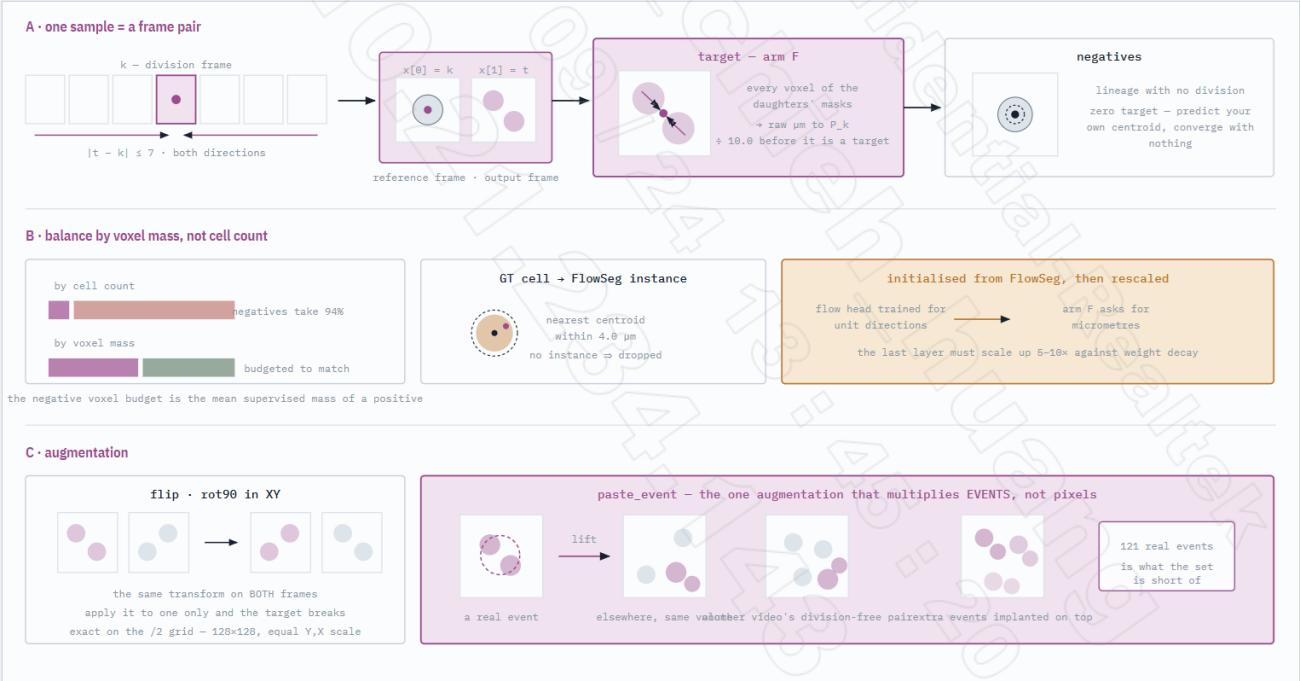
Both conditions are pure distance tests on quantities the network produced; no decision head is involved.

- Why this framing is the strong one: It converts a rare-event classification problem into a dense regression problem: every voxel of every supervised cell contributes gradient, rather than one label per event. And the two conditions are independent enough to catch each other's failure modes. One cell over-segmented into two fragments converges — but the fragments are closer than 3 µm, so it is rejected. A flow error that maps two unrelated cells to the same place converges — but they are further apart than 16 µm, so it is rejected. Neither test alone would be trustworthy; together they are, because they fail in different directions.





- Training: A sample is a frame pair (k, t) with $|t - k| \leq 7$ in both directions — channel 0 is the division frame the target is expressed in, channel 1 is where the output lives. The shipped arm regresses a raw micrometre displacement toward the division centre, supervised only inside the daughters' instance masks, scaled down by 10 before it becomes a target. Negatives are cells whose entire lineage contains no division, given a zero target — a cell that predicts its own centroid converges with nothing, which is exactly what "no division" means. Because the loss is voxel-weighted, balance is by voxel mass rather than cell count; by cell count the negatives would take 94% of the gradient. And since the scarce resource is events, not pixels, the one augmentation that matters lifts a whole division event and pastes it elsewhere — into another region, into a different video's division-free frame pair, or stacked several to a volume.
- What it deliberately is not: The output is a vote, not a division. Clustering, a parent search, and the xgboost / catboost / verifier filters all sit downstream. divflow's job is to propose geometrically defensible candidates cheaply and densely; deciding which survive is someone else's.

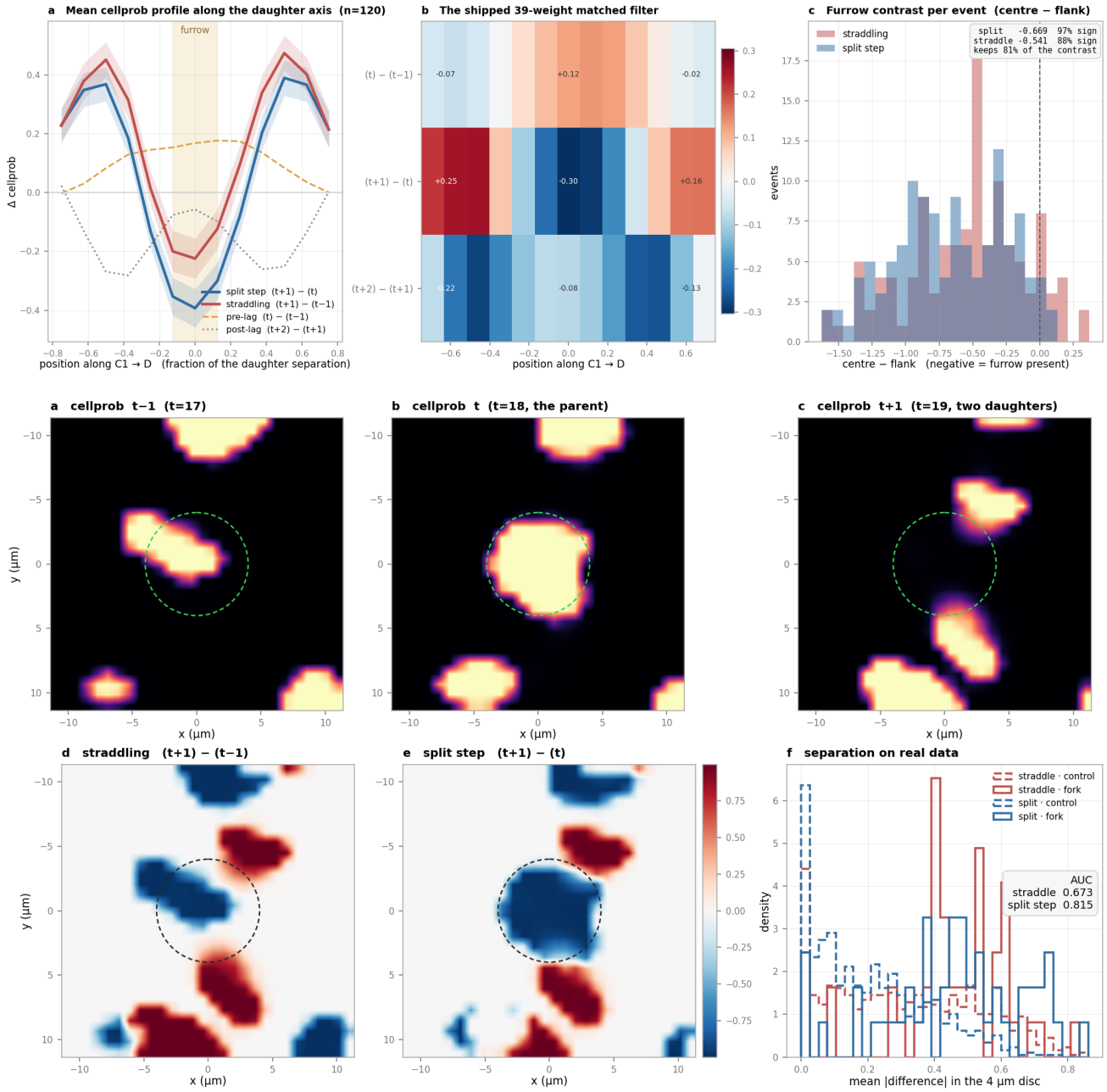


Safe-division repair, with a learned veto

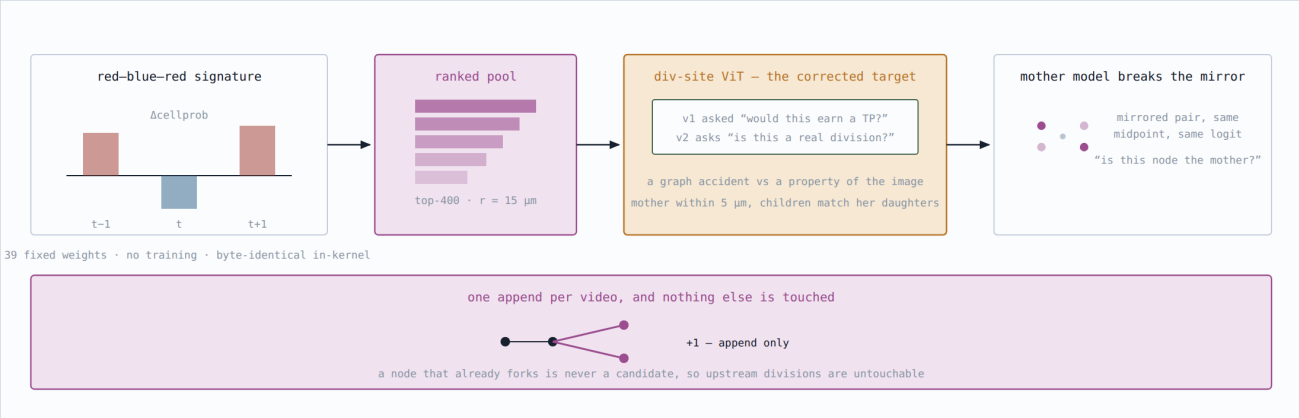
- The problem: the ILP emits no forks at all, so something has to propose them.
- The idea: propose on geometry, then veto with a model. A track end sitting close to the start of two other tracks is a candidate; the two candidate daughters must diverge from the mother and must be roughly balanced about her, with caps on how many may be added per frame and per video so a single bad video cannot flood the graph.
- The veto: each surviving candidate is scored by a centre-prior network and rejected below threshold. A real run's log makes the ratio explicit — 176 geometric candidates, 142 rejected by the veto, 34 surviving, 32 added. The geometry is deliberately generous and the model does the discriminating.
- Why this order: geometry is cheap and has nothing to overfit; the model is expensive and has very few events. Proposing with the cheap one and filtering with the expensive one uses each where it is strong.

The RBR adder and the div-site ViT

- The problem: by the end of the chain, the divisions still missing are the ones no geometric rule proposed. They need image evidence.
- The idea — a red–blue–red prior: a dividing nucleus passes through a characteristic intensity signature across three time points. We compute it as a fixed 39-weight oriented prior over Δcellprob at several lags — no training, and byte-identical offline and in the kernel — producing a ranked pool of candidate sites.
- The selector, and the target that had to be corrected: a ViT re-ranks that pool. Its first version was trained on "would inserting here earn a division true positive?", which turned out to be a property of the graph rather than of the image — two of the four TP-causing insertions scored below the FP median. The shipped model is trained on the div-site label instead: does this candidate sit on a real division, with its mother within 5 µm of an annotated mother and its children matching her daughters. That is a question about the image, and it is answerable from the image.

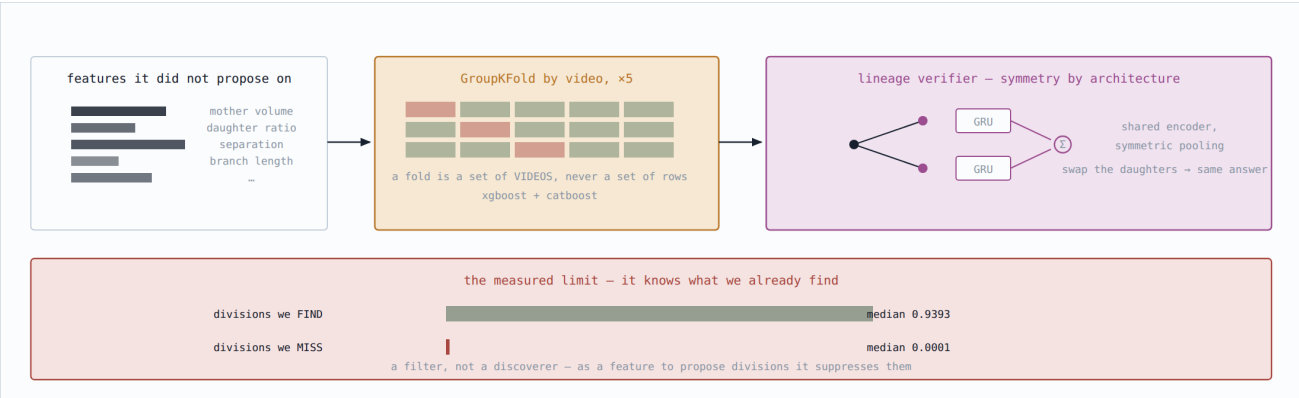


- The mother model, and why it exists: every signal centred on the daughter pair is reflection-symmetric, so a mirrored pair of candidates shares a midpoint and the div-site ViT gives both the same logit. A separate model asks "is this node a division mother at t → t+1", which breaks the symmetry and gives the layer a direction.
- How it predicts: at most one append per video, append-only. Nothing is ever deleted and no node that already forks is touched, so every division predicted upstream is untouchable by construction rather than by convention.



Division classifiers and the lineage verifier

- The problem: a proposed fork can be checked against features nobody used to propose it — the volumes of mother and daughters, their separation, how long each branch survives.
- The tabular models: xgboost and catboost, five folds each, GroupKFold **by video**. Grouping by video rather than by row is not a detail — two candidates from the same video share a detector, a frame and an embryo, and a row-wise split reports a score the leaderboard never reproduces.
- The lineage verifier: a deliberately tiny GRU over the family tree, a few thousand parameters rather than a few million, because the corpus holds about 151 annotated divisions. Its daughter-swap symmetry is imposed architecturally, through a shared encoder and symmetric pooling, rather than by augmentation — that halves the effective data requirement and removes a whole class of spurious learning. GroupKFold by embryo is treated as the only honest split.
- The measured limit, stated because it changed how we used it: the verifier's median score on the divisions the chain misses is 0.0001, against 0.9393 overall. It knows the divisions we already find. It is a filter, not a discoverer, and used as a feature to propose divisions it actively suppresses them.



Interesting Findings

The ILP never divides. With the shipped weights a fork gains 1.0 and costs 1.2, so it is priced out entirely and the linker contributes zero divisions. Every division in the submission is created downstream. Knowing this changed where we spent the rest of the competition.

Balance by voxel mass, not cell count. In a voxel-weighted loss, counting cells hands the negatives 94% of the gradient. This is the difference between divflow training and divflow collapsing, and it is invisible until you print the realised mass per epoch.

A zero median does not mean no effect. Our post-hoc shift had a per-video median of exactly zero on the local harness and still returned +0.002 on the leaderboard. We now judge a candidate on its per-video win rate and its spread, not on whether the median moved.

Localisation helps the scorer and hurts the graph. The same coordinate correction is +0.01116 at the matching step and −0.00573 to the linker, because moving detections before linking costs 73 true edges. Anything that moves detections has to be priced on both sides, and applied where only the good half lands.

Out-of-fold can understate a shipped model. An out-of-fold score vector is a mixture of k differently-calibrated models, and a single absolute threshold cuts it at k operating points — which smears exactly the head of the ranking, where a deletion or selection layer lives. Retrained on a disjoint set of videos and applied once, the same layer measured materially better than its own OOF estimate.

A verifier only knows what you already find. Median 0.9393 on the divisions we detect, 0.0001 on the ones we miss. It is a filter. Using it as a feature to propose divisions actively suppresses them.

Things we deliberately did not ship. A steal layer and an insertion layer, both of which moved the division term but delete or insert structure, making every number conditional on the rest of the chain being exactly as measured. A second divflow offset, which costs 0.7–1.05 h of the runtime budget and changes the score denominator of every single-offset candidate. And a learned appearance model for linking, which did not transfer across the two embryo families in this data.